# AI chatbots with shinychat (R) : : CHEAT SHEET

## Create a basic chatbot

**1. Connect to LLM**

Choose model provider and provide instructions

**2. Add capabilities**

Register tools like web search

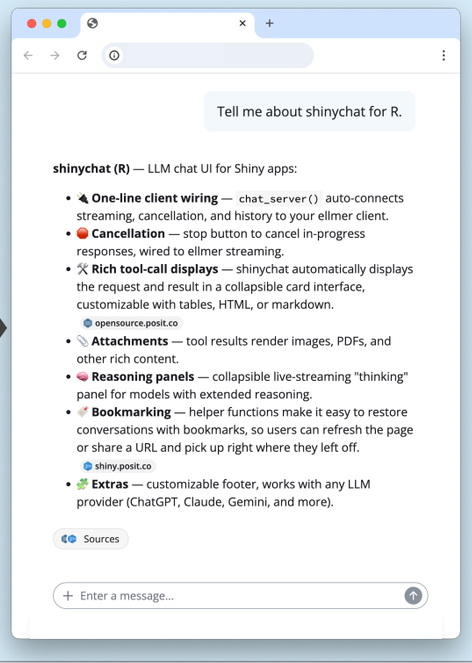
**3.Create front-end**

Create and display the `Chat .ui()`

```
# app.R
library(shiny)
library(shinychat)

client <- ellmer::chat_anthropic(
  model = "claude-sonnet-5",
  system_prompt = "You are a helpful assistant."
)
client$register_tool(ellmer::claude_tool_web_search())
client$register_tool(ellmer::claude_tool_web_fetch())

ui <- bslib::page_fillable(
  chat_ui("chat", greeting = "Welcome to my chat!")
)
server <- function(input, output, session) {
  chat_server("chat", client)
}
shinyApp(ui, server)
```




## Chat features

**BUILT-IN**

With no extra effort, get the following

- **History:** archive, revisit, and share conversations.
- **Attachments:** include images, pdfs, plain text, & more.
- **Cancel & edit:** input the wrong prompt? Cancel, revise and resubmit.
- **Tool displays:** let users inspect, validate, and learn from code the model executes.
- **Thinking display:** inspect how LLMs reason through hard problems.
- **Rich streaming output:** performantly stream LLM (markdown) responses with rich formatting for code blocks, images, and more.

**OPT-IN**

With extra effort, add the following

- **Greetings:** supply a useful intro to the chat
- **Suggestions:** make it easy to get started
- **Slash commands:** useful shortcuts for common workflows

## Connect to LLM

### CHOOSE A PROVIDER

Initialize an LLM chat provider via `ellmer`.

Anthropic Claude: **`chat_claude()`**
OpenAI: **`chat_openai()`**
Google Gemini: **`chat_google()`**
AWS Bedrock: **`chat_aws_bedrock()`**
Azure OpenAI: **`chat_azure_openai()`**
Databricks: **`chat_databricks()`**
Posit AI: **`chat_posit()`**
LMStudio: **`chat_lmstudio()`**

Many require an API key. Provide yours to `edit_r_environ()` from `usethis`.

Visit https://ellmer.tidyverse.org/reference/index.html to see **all available** providers and their **prerequisites**.

### QUICKLY ITERATE

Launch a chat app with one line

```
client = chat_claude()
live_browser(client)
```

### STREAM OUTPUT ANYWHERE

Not only in Shiny, but at the console, etc.

```
client = chat_claude()
client$chat("Hello!")
```

## First impression

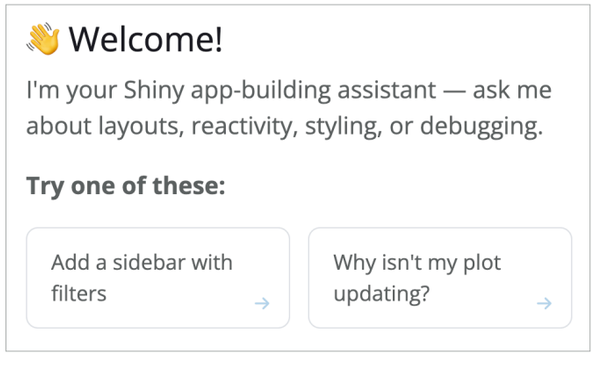
### GREET & SUGGEST

Customize what users see on first page load.

Supply markdown/HTML for a custom `greeting`. Greetings can include special HTML suggestion markup.

```
chat_ui(greeting="""
### 👋 Welcome!

Not sure what to do? Try:
1. <span class="suggestion">Tell me a joke</span>
2. <span class="suggestion">Write a poem</span>
""")
```




## Layouts

### LOCATE THE CHAT IN A LARGER SHINY APP

Use shiny/bslib to customize layout and functionality

**SCREEN-FILLING LAYOUT**

```
ui <- page_fillable(
  chat_ui("chat"),
  fillable_mobile = TRUE
)
```

Make contents (chat) **fill page**

Also fill on mobile

**SIDEBAR LAYOUT**

```
ui <- page_sidebar(
  sidebar = sidebar(
    chat_ui("chat"),
    fillable = TRUE
  )
)
```

Make contents (chat) **fill sidebar**

**CARD LAYOUT**

```
ui <- page_fillable(
  card(
    chat_ui(id = "chat")
  )
)
```

Make contents (card) fill **page**

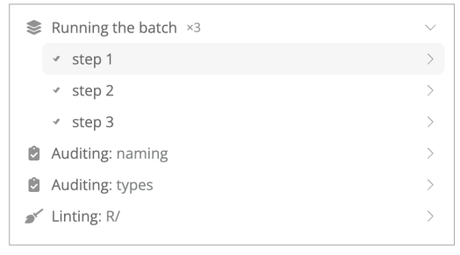
Cards are fillable by default

## Extras

### Tool Displays

Inform users of actions taken by the LLM

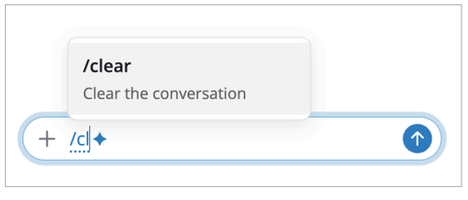
Tool call information is automatically displayed to the user (for transparency). The display is also fully customizable.




### Slash commands

Add shortcuts for common workflows

```
chat$slash_command("clear", "Clear", \(){
  chat$clear()
})
```



# AI chatbots with shinychat (**Python**) : : CHEAT SHEET

## Create a basic chatbot

**1. Connect to LLM**

Choose model provider and provide instructions

**2. Add capabilities**

Register tools like web search

**3.Create front-end**

Create and display the `Chat .ui()`

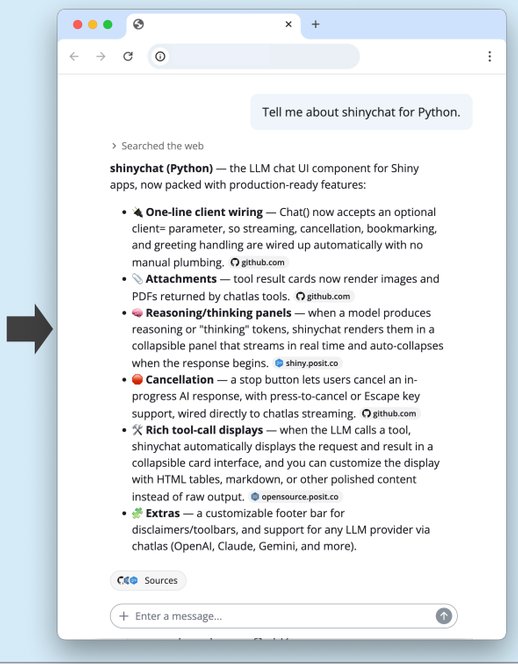
```
# app.py
import chatlas as ctl
from shinychat.express import Chat

client = ctl.ChatAnthropic(
  model="claude-sonnet-5",
  system_prompt="You are a helpful assistant."
)

client.register_tool(ctl.tool_web_search())
client.register_tool(ctl.tool_web_fetch())

chat = Chat(id="chat", client=client)
chat.ui(greeting="Welcome to my custom chat!")
```

This cheatsheet uses **Shiny Express**. To see examples for Shiny Core, see https://shiny.posit.co/py/docs/genai-chatbots.html.




## Chat features

### BUILT-IN

With no extra effort, get the following

- **History:** archive, revisit, and share conversations.
- **Attachments:** include images, pdfs, plain text, & more.
- **Cancel & edit:** input the wrong prompt? Cancel, revise and resubmit.
- **Tool displays:** let users inspect, validate, and learn from code the model executes.
- **Thinking display:** inspect how LLMs reason through hard problems.
- **Rich streaming output:** performantly stream LLM (markdown) responses with rich formatting for code blocks, images, and more.

### OPT-IN

With extra effort, add the following

- **Greetings:** supply a useful intro to the chat
- **Suggestions:** make it easy to get started
- **Slash commands:** useful shortcuts for common workflows

## Connect to LLM

### CHOOSE A PROVIDER

Initialize an LLM chat provider via `chatlas`.

Anthropic Claude: **`ChatAnthropic()`**
OpenAI: **`ChatOpenAI()`**
Google Gemini: **`ChatGoogle()`**
AWS Bedrock: **`ChatBedrock()`**
Azure OpenAI: **`ChatAzureOpenAI()`**
Databricks: **`ChatDatabricks()`**
Posit AI: **`ChatPosit()`**
LMStudio: **`ChatLMStudio()`**

Many require an API key. Place yours in **`.env`** file and load with **`dotenv`**.

Visit https://posit-dev.github.io/chatlas/reference to see **all available** providers and their **prerequisites**.

### QUICKLY ITERATE

Launch a chat app with one line

```
client = ctl.ChatAnthropic()
client.app()
```

### STREAM OUTPUT ANYWHERE

Not only in Shiny, but in notebooks, the console, etc.

```
client = ctl.ChatAnthropic()
client.chat("Hello!")
```

## First impression

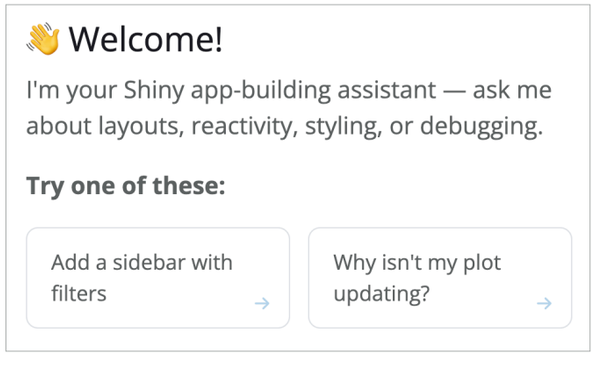
### GREET & SUGGEST

Customize what users see on first page load.

Supply markdown/HTML for a custom `greeting`. Greetings can include special HTML suggestion markup.

```
chat.ui(greeting="""
### 👋 Welcome!

Not sure what to do? Try:
1. <span class="suggestion">Tell me a joke</span>
2. <span class="suggestion">Write a poem</span>
""")
```




## Layouts

### LOCATE THE CHAT IN A LARGER SHINY APP

Use shiny to customize layout and functionality

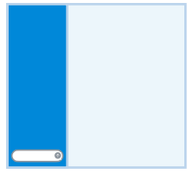
**SCREEN-FILLING LAYOUT**

```
ui.page_opts(
  fillable=True,
  fillable_mobile=True,
)

chat = ui.Chat(id="chat")
chat.ui()
```

Make contents (Chat) **fill page**

**SIDEBAR LAYOUT**



```
chat = ui.Chat(id="chat")
with ui.sidebar(fillable=True):
  chat.ui()

"Main content"
```

Make contents (chat) **fill sidebar**

**CARD LAYOUT**

```
ui.page_opts(
  fillable=True,
  fillable_mobile=True
)

chat = ui.Chat(id="chat")

with ui.card():
    chat.ui()
```

Make contents (card) fill **page**

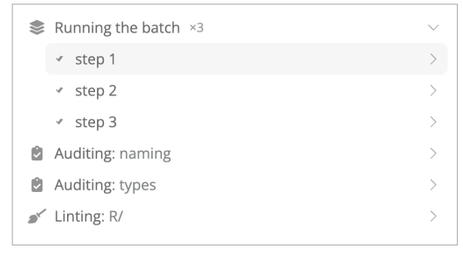
Cards are fillable by default

## Extras

### Tool Displays

Inform users of actions taken by the LLM

Tool call information is automatically displayed to the user (for transparency). The display is also fully customizable.




### Slash commands

Add shortcuts for common workflows

```
@chat.slash_command("clear", "Clear")
async def _():
    await chat.clear()
```